```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

# 1. Load Data & Handle Missing Values
print("Handling Missing Values")

df = pd.read_csv("Dataset_Day9.csv")
data = df.copy()

missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
for col in missing_cols:
    data[col] = data[col].replace(0, np.nan)
    median_val = data[col].median()
    print(f"Replaced 0s in '{col}' with median: {median_val}")
    data[col] = data[col].fillna(median_val)

# 2. Remove Outliers using Z-score

features = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
data_z = data[features]
z_scores = (data_z - data_z.mean()) / data_z.std()
outliers = (np.abs(z_scores) > 3).any(axis=1)
outlier_count = outliers.sum()
data_cleaned = data[~outliers]

print(f"Outliers removed: {outlier_count}")
print(f"Remaining rows after outlier removal: {data_cleaned.shape[0]}\n")
```

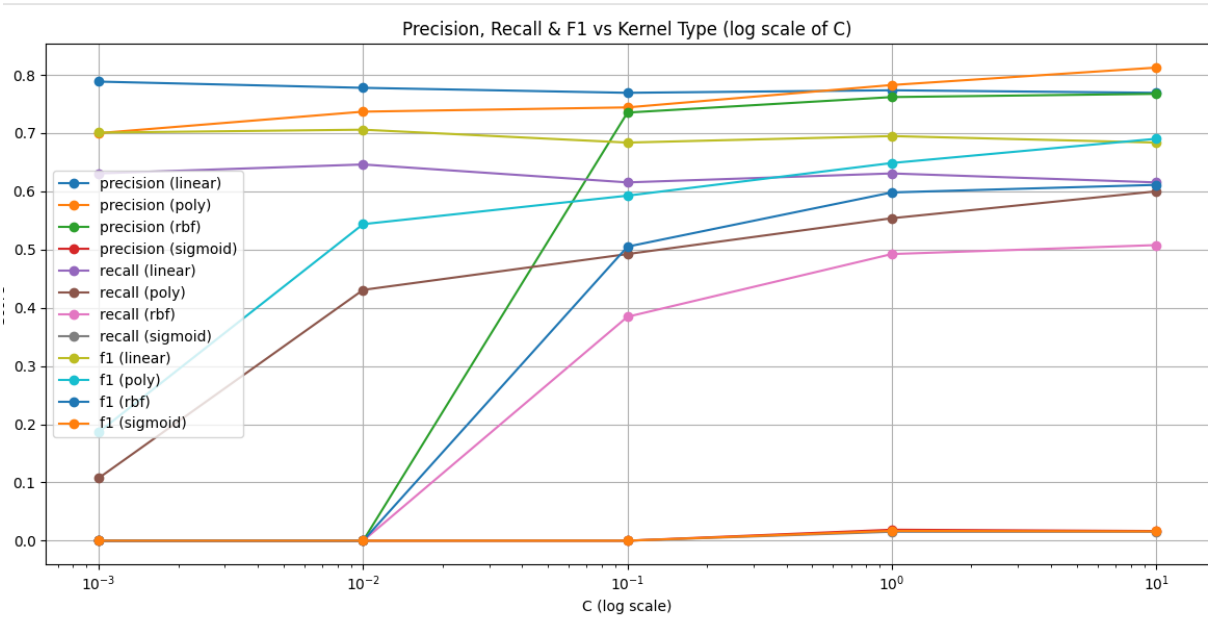
```
(myenv) flora_04@Flora:/mnt/d/python/day9$ python3 solution.py
Handling Missing Values
Replaced 0s in 'Glucose' with median: 117.0
Replaced 0s in 'BloodPressure' with median: 72.0
Replaced 0s in 'BMI' with median: 32.3
Replaced 0s in 'DiabetesPedigreeFunction' with median: 0.3725
Replaced 0s in 'Age' with median: 29.0
Outliers removed: 26
Remaining rows after outlier removal: 742
```

```python
# 3. SVM Classification
X = data_cleaned.drop("Outcome", axis=1)
y = data_cleaned["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
svm_default = SVC()
svm_default.fit(X_train, y_train)
y_pred = svm_default.predict(X_test)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(" Default SVM Metrics (kernel='rbf', C=1.0):")
print(f" Accuracy : {acc:.4f}")
print(f" Precision: {prec:.4f}")
print(f" Recall   : {rec:.4f}")
print(f" F1 Score : {f1:.4f}\n")

# 3b. Compare Kernels Across C Values
kernel_types = ['linear', 'poly', 'rbf', 'sigmoid']
C_values = [0.001, 0.01, 0.1, 1, 10]
results = []
for kernel in kernel_types:
    for c in C_values:
        svm = SVC(kernel=kernel, C=c)
        svm.fit(X_train, y_train)
        y_pred = svm.predict(X_test)
        results.append({
            'kernel': kernel,
            'C': c,
            'precision': precision_score(y_test, y_pred, zero_division=0),
            'recall': recall_score(y_test, y_pred, zero_division=0),
            'f1': f1_score(y_test, y_pred, zero_division=0)
        })

results_df = pd.DataFrame(results)
```

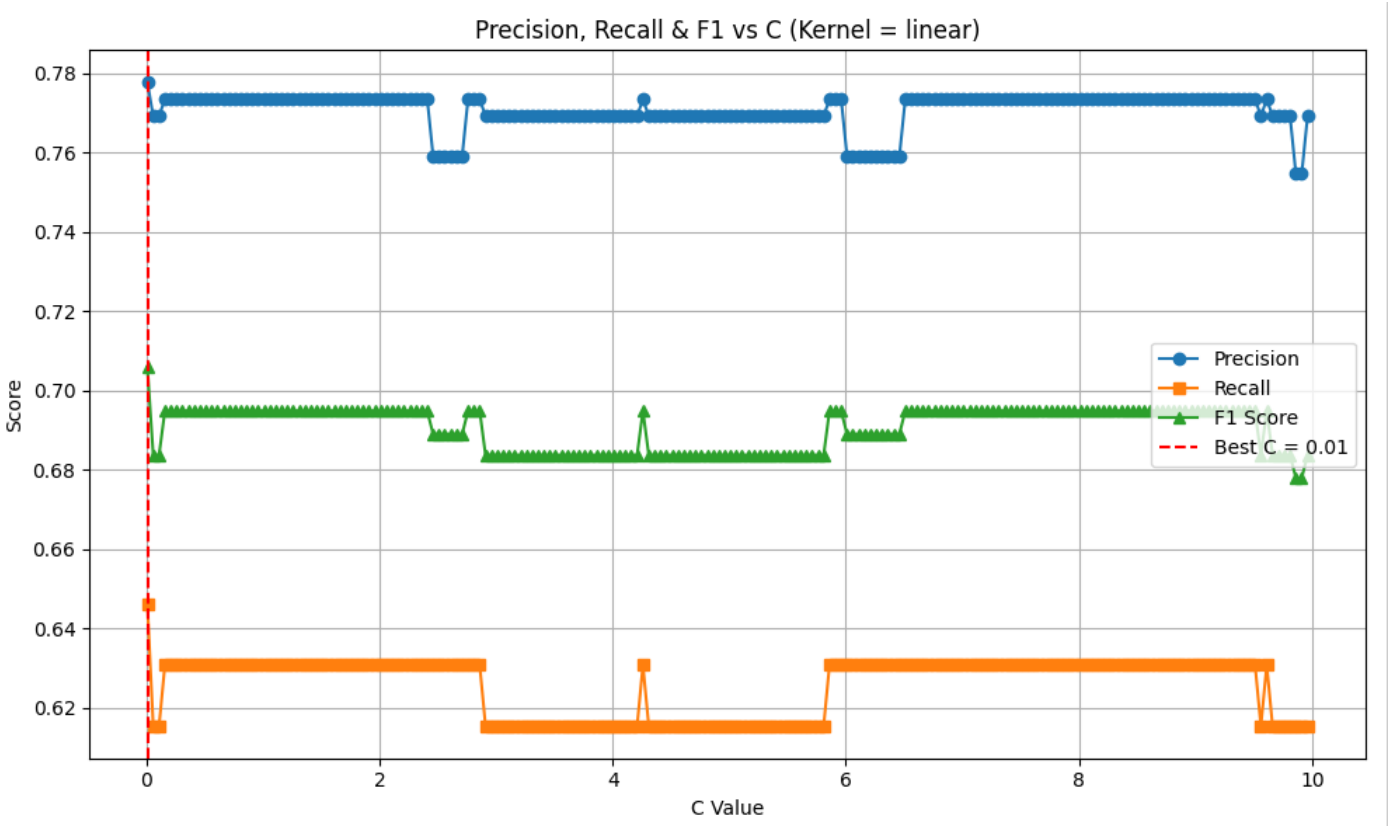
```
Default SVM Metrics (kernel='rbf', C=1.0):
Accuracy : 0.7688
Precision: 0.7619
Recall   : 0.4923
F1 Score : 0.5981
```

```python
# Plot metrics per kernel
plt.figure(figsize=(12, 6))
for metric in ['precision', 'recall', 'f1']:
    for kernel in kernel_types:
        subset = results_df[results_df['kernel'] == kernel]
        plt.plot(subset['C'], subset[metric], marker='o', label=f'{metric} ({kernel})')

plt.xscale('log')
plt.xlabel('C (log scale)')
plt.ylabel('Score')
plt.title('Precision, Recall & F1 vs Kernel Type (log scale of C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Identify best kernel based on F1-score
best_row = results_df.loc[results_df['f1'].idxmax()]
best_kernel = best_row['kernel']
```

```python
# 3c. Fine-tune C for Best Kernel

C_range = np.arange(0.01, 10.01, 0.05)
prec_list, rec_list, f1_list = [], [], []

for c in C_range:
    svm = SVC(kernel=best_kernel, C=c)
    svm.fit(X_train, y_train)
    y_pred = svm.predict(X_test)

    prec_list.append(precision_score(y_test, y_pred))
    rec_list.append(recall_score(y_test, y_pred))
    f1_list.append(f1_score(y_test, y_pred))

best_c_final = C_range[np.argmax(f1_list)]
best_f1_final = max(f1_list)

# Plot performance vs C for best kernel
plt.figure(figsize=(10, 6))
plt.plot(C_range, prec_list, label='Precision', marker='o')
plt.plot(C_range, rec_list, label='Recall', marker='s')
plt.plot(C_range, f1_list, label='F1 Score', marker='^')
plt.axvline(x=best_c_final, color='r', linestyle='--', label=f'Best C = {best_c_final:.2f}')
plt.xlabel('C Value')
plt.ylabel('Score')
plt.title(f'Precision, Recall & F1 vs C (Kernel = {best_kernel})')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f"Best C for kernel='{best_kernel}':")
print(f" C Value    = {best_c_final:.2f}")
```




```
Best Kernel = 'linear' at C = 0.01 with F1-Score = 0.7059
```




```
Best C for kernel='linear':
 C Value    = 0.01
```